	Politechnika Bydgoska im. J. J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki		
Przedmiot	Zaawansowane programowanie obiektowe		
Prowadzący	dr inż. Damian Szczegielniak		
Temat	Implementacja walidatora danych z wykorzystaniem adnotacji, refleksji i wzorca strategia		
Student	Jakub Ziółkowski, grupa 1, Informatyka stosowana, IV semestr, studia stacjonarne		
Nr ćw.	2	Data wykonania	25.04.2026 r.
Ocena		Data oddania spr.	29.04.2026 r.

Spis treści

1. Cel laboratorium.....	2
2. Rozwiązanie	3
2.1. Ogólny opis aplikacji	3
2.2. Struktura projektu i podział na pakiety	3
2.3. Klasa Student i opis pól poddawanych walidacji	4
2.4. Implementacja własnych adnotacji walidacyjnych	5
2.5. Interfejs ValidationStrategy i idea wzorca Strategia	6
2.6. Implementacja klas strategii dla zadania podstawowego	6
2.6.1. NotNullStrategy	7
2.6.2. NotEmptyStrategy	7
2.6.3. SizeStrategy	7
2.6.4. NrIndeksuStrategy	8
2.6.5. EmailStrategy.....	8
2.7. Klasa ValidationStrategyFactory w zadaniu podstawowym	8
2.8. Klasa ValidationException i obsługa błędów walidacji	9
2.9. Klasa Validator i wykorzystanie refleksji	9
2.9.1. Pobieranie pól klasy.....	10
2.9.2. Udostępnianie pól prywatnych	10

2.9.3. Iteracja po adnotacjach	10
2.9.4. Gromadzenie komunikatów błędów	11
2.10. Testowanie rozwiązania podstawowego w klasie Main	12
2.11. Zadanie dodatkowe - założenie i cel	12
2.11.1 Adnotacja @ValidationFor	13
2.11.2 Oznaczanie strategii adnotacją @ValidationFor	13
2.11.3 Automatyczne wyszukiwanie strategii w ValidationStrategyFactory	14
2.11.4 Biblioteka Reflections i konfiguracja projektu dodatkowego	15
2.11.5. Problemy konfiguracyjne w Eclipse i ich rozwiązanie	16
3. Wnioski końcowe/Podsumowanie	17

1. Cel laboratorium

Celem laboratorium było opracowanie mechanizmu walidacji danych wejściowych w języku Java z wykorzystaniem własnych adnotacji, refleksji oraz wzorca projektowego Strategia. Zgodnie z instrukcją rozwiązanie miało umożliwiać automatyczne sprawdzanie poprawności danych obiektów, których pola zostały oznaczone odpowiednimi adnotacjami walidacyjnymi, bez konieczności pisania osobnej logiki walidacyjnej dla każdej klasy modelu.

W praktyce oznaczało to przygotowanie zestawu własnych adnotacji, klas strategii realizujących sprawdzanie poszczególnych ograniczeń, fabryki udostępniającej odpowiednie strategie oraz klasy walidatora, która przy użyciu refleksji analizuje pola obiektu i uruchamia właściwe reguły walidacji.

Zrealizowałem również zadanie dodatkowe z instrukcji, którego celem było wyeliminowanie konieczności ręcznego deklarowania w kodzie powiązań pomiędzy adnotacjami walidacyjnymi a klasami strategii oraz zaimplementowanie mechanizmu automatycznego wyszukiwania strategii i ich rejestrowania w fabryce.

2. Rozwiązanie

2.1. Ogólny opis aplikacji

W ramach laboratorium przygotowałem dwa osobne projekty w Eclipse. Pierwszy projekt realizuje zadanie podstawowe, a więc mechanizm walidacji pól obiektu z ręcznie skonfigurowaną fabryką strategii. Drugi projekt realizuje zadanie dodatkowe i rozszerza pierwsze rozwiązanie o automatyczne wykrywanie strategii na podstawie adnotacji @ValidationFor.

W obu wariantach zadania najważniejsza jest klasa Student. Pola tej klasy są oznaczane adnotacjami walidacyjnymi: @NotNull, @NotEmpty, @Size, @NrIndeksu oraz @Email, a następnie poddawane analizie przez klasę Validator.

W klasie Main tworzony jest obiekt Student, przypisywane są mu wybrane wartości testowe, po czym uruchamiana jest walidacja. Jeśli w danych wystąpią błędy, program zgłasza wyjątek ValidationException, którego komunikat zostaje wypisany w konsoli.

Adnotacje odpowiadają za deklaracyjny opis reguł, strategie przechowują logikę walidacyjną, fabryka zapewnia dostęp do właściwych strategii, a walidator łączy wszystko przy użyciu refleksji. Taki podział zwiększa czytelność kodu.

2.2. Struktura projektu i podział na pakiety

Strukturę obu projektów oparłem o przykładowy układ pokazany w instrukcji. W projektach występują następujące pakiety:

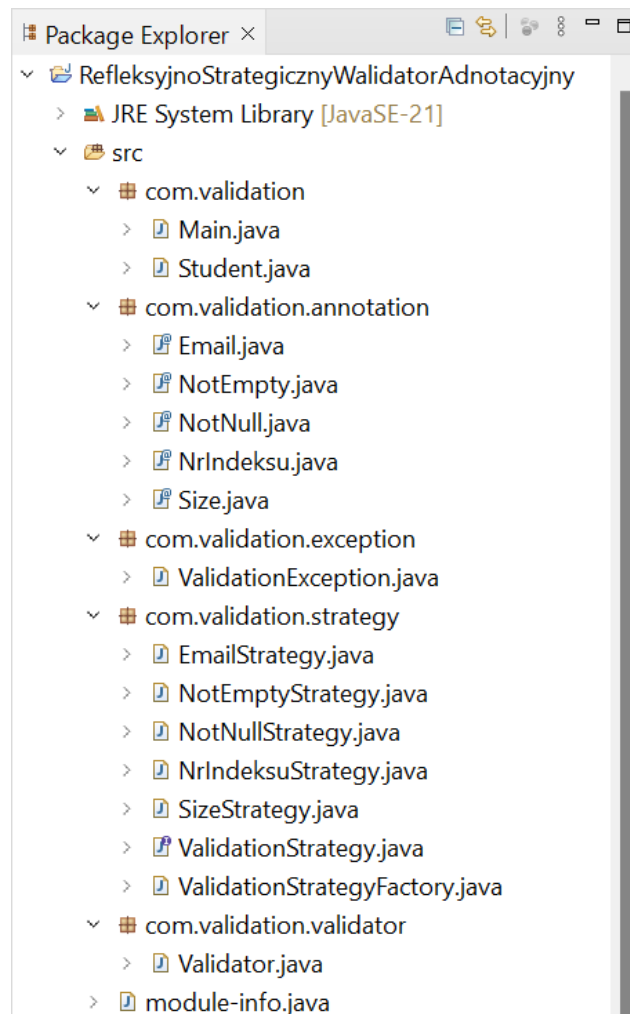
- com.validation,
- com.validation.annotation,
- com.validation.exception
- com.validation.strategy,
- com.validation.validator

Klasy Student i Main (z pakietu com.validation) są wykorzystywane do przechowywania danych testowych i uruchamiania programu.

Pakiet com.validation.annotation zawiera definicje wszystkich własnych adnotacji. Dzięki temu reguły walidacyjne są zebrane w jednym miejscu i można je wielokrotnie stosować. Pakiet com.validation.strategy przechowuje interfejs strategii oraz wszystkie klasy implementujące konkretne reguły sprawdzania danych.

W pakiecie com.validation.exception umieszczona została klasa ValidationException, która odpowiada za sygnalizowanie błędów walidacji. Z kolei pakiet com.validation.validator zawiera klasę Validator, czyli najważniejszy element wykonawczy całego rozwiązania.

W projekcie dodatkowym dodałem jeszcze katalog lib, w którym umieściłem biblioteki JAR potrzebne do działania mechanizmu automatycznego skanowania klas strategii. Sama struktura kodu źródłowego pozostała jednak bardzo podobna do wersji podstawowej, przy czym, ze względu na dodanie bibliotek JAR do projektu, zrezygnowałem z pliku module-info.java w projekcie.



Rysunek 1 - Package Explorer z projektem podstawowym - struktura projektu

2.3. Klasa Student i opis pól poddawanych walidacji

Klasa Student zawiera pola imie, nazwisko, nrIndeksu oraz email, czyli dane tekstowe, dla których można zastosować różne ograniczenia logiczne i formalne, testując mechanizm walidacji.

Pole imie zostało opisane adnotacjami @NotNull i @Size, co oznacza, że wartość nie może być null, a dodatkowo musi spełniać limity (min, maks) długości. Analogicznie wygląda pole nazwisko, dla którego również jest sprawdzana wartość (@NotNull) oraz dopuszczalny przedział liczby znaków.

Pole nrIndeksu zostało oznaczone adnotacjami @NotNull i @NrIndeksu. W tym przypadku oprócz obecności wartości, wymagany jest odpowiedni format numeru indeksu, czyli ośmiocyfrowy łańcuch znaków.

Pole email wykorzystuje adnotacje @NotEmpty oraz @Email. Oznacza to, że adres e-mail nie może być pusty (empty), a jednocześnie musi spełniać warunek poprawnej składni (według wyrażenia regularnego).

2.4. Implementacja własnych adnotacji walidacyjnych

Wszystkie własne adnotacje walidacyjne utworzyłem w pakiecie com.validation.annotation. Każda adnotacja została przygotowana tak, aby mogła oznaczać pola i aby była dostępna w trakcie działania programu.

Wspólną cechą wszystkich adnotacji jest zastosowanie RetentionPolicy.RUNTIME. Jest to konieczne, ponieważ klasa Validator odczytuje informacje o adnotacjach dopiero podczas wykonywania programu, przy użyciu refleksji. Gdyby użyć innego rodzaju retencji, dane o adnotacjach nie byłyby dostępne na etapie działania aplikacji.

Drugim ważnym elementem jest użycie @Target(ElementType.FIELD). Oznacza to, że adnotacje zostały zaprojektowane do oznaczania pól klasy, czyli miejsca, w którym znajdują się dane poddawane walidacji.

Każda adnotacja pełni rolę deklaracji reguły. Sama adnotacja nie zawiera jeszcze logiki sprawdzania danych, ale przekazuje metadane, które potem odczytuje odpowiednia strategia.

Wszystkie adnotacje w projekcie zadania podstawowego zawierają parametr message, czyli komunikat błędu. Dzięki niemu można przypisać do każdego pola bardziej precyzyjny opis, zamiast korzystać z jednego ogólnego komunikatu dla wszystkich przypadków.

```
1 package com.validation.annotation;
2
3 import java.lang.annotation.ElementType;
4 import java.lang.annotation.Retention;
5 import java.lang.annotation.RetentionPolicy;
6 import java.lang.annotation.Target;
7
8 @Retention(RetentionPolicy.RUNTIME)
9 @Target(ElementType.FIELD)
10 public @interface NotNull {
11     String message() default "Pole wymaga wartości - nie może być null";
12 }
```

Rysunek 2 - kod przykładowej adnotacji @NotNull (NotNull.java) w pakiecie com.validation.annotation

2.5. Interfejs ValidationStrategy i idea wzorca Strategia

Instrukcja laboratoryjna wymagała użycia wzorca projektowego Strategia do zarządzania logiką walidacji przypisaną do poszczególnych adnotacji. W praktyce oznacza to, że każda reguła walidacyjna ma własną klasę odpowiedzialną tylko za jeden rodzaj sprawdzenia.

Podstawą tego mechanizmu jest interfejs ValidationStrategy. Definiuje on jedną metodę, która przyjmuje dwa argumenty: obiekt Field, reprezentujący pole klasy, oraz aktualną wartość tego pola. Wynikiem jest Optional<String>, czyli opcjonalny komunikat o błędzie.

Taki interfejs wymusza jednolity sposób działania wszystkich strategii. Dzięki temu klasa Validator może wywoływać każdą strategię w taki sam sposób, niezależnie od tego, czy sprawdza ona, czy pole jest not null, długość tekstu, numer indeksu czy składnię e-maila.

Wzorec projektowy Strategia pasuje do tego problemu. Poszczególne reguły są od siebie niezależne, a ich liczba może rosnąć wraz z rozwojem systemu. Zamiast umieszczać całą logikę w jednej klasie z dużą liczbą instrukcji warunkowych, każda reguła jest zamknięta osobno.

2.6. Implementacja klas strategii dla zadania podstawowego

W projekcie podstawowym dla każdej adnotacji została utworzona osobna klasa strategii. Wszystkie te klasy znajdują się w pakiecie com.validation.strategy i implementują interfejs ValidationStrategy.

Taki układ pozwala podzielić walidację na małe, wyspecjalizowane moduły. Ponadto, kod jest bardziej czytelny, ponieważ każda klasa odpowiada za jeden jasno określony warunek. Wreszcie, testowanie i rozwijanie poszczególnych strategii niezależnie od reszty jest dużo łatwiejsze.

```
1 package com.validation.strategy;
2
3 import java.lang.reflect.Field;
4 import java.util.Optional;
5 import com.validation.annotation.NotNull;
6
7 public class NotNullStrategy implements ValidationStrategy {
8
9     @Override
10    public Optional<String> validate(Field field, Object value) {
11        if (field.isAnnotationPresent(NotNull.class) && value == null) {
12            NotNull annotation = field.getAnnotation(NotNull.class);
13            String errorInfo = String.format("Pole %s: %s", field.getName(), annotation.message());
14            return Optional.of(errorInfo);
15        }
16        return Optional.empty();
17    }
18 }
```

Rysunek 3 - kod przykładowej strategii NotNullStrategy (NotNullStrategy.java) w pakiecie com.validation.strategy

Poniżej opisane zostały najważniejsze klasy strategii zastosowane w rozwiązaniu.

2.6.1. NotNullStrategy

Klasa NotNullStrategy odpowiada za sprawdzanie, czy wartość pola nie jest null. Implementacja odczytuje z pola adnotację @NotNull, pobiera z niej komunikat i zwraca informację o błędzie wtedy, gdy pole nie ma wartości.

Jest to najprostsza strategia w całym projekcie, ale dobrze pokazuje ogólny schemat działania wszystkich pozostałych klas. Strategia nie musi znać całej klasy Student, ale interesuje ją aktualna wartość wyłącznie jednego pola oraz metadane zapisane w adnotacji.

2.6.2. NotEmptyStrategy

Klasa NotEmptyStrategy sprawdza, czy wartość tekstowa nie jest pusta. W odróżnieniu od NotNullStrategy nie wystarcza tu samo sprawdzenie braku obiektu - trzeba upewnić się, że tekst w polu zawiera jakąś treść.

Strategia ta jest szczególnie przydatna przy polu email, ponieważ użytkownik może wstawić pusty łańcuch znaków zamiast rzeczywistego adresu. W takim przypadku pole formalnie istnieje, ale nie spełnia wymagań logicznych aplikacji.

Rozdzielenie NotNull i NotEmpty na dwie niezależne strategie pozwala dokładniej opisywać błędy.

2.6.3. SizeStrategy

Klasa SizeStrategy realizuje walidację długości tekstu. Odczytuje z adnotacji wartości minimalnej i maksymalnej długości, a następnie sprawdza, czy rzeczywista długość tekstu mieści się w zadanym przedziale.

Strategia ta przydaje się zwłaszcza przy polach imienia i nazwiska, gdzie należy wykreślić zarówno wartości zbyt krótkie, jak i zbyt długie.

Dzięki wykorzystaniu parametrów min i max jedna strategia może obsługiwać różne pola z różnymi ograniczeniami.

2.6.4. NrIndeksuStrategy

Klasa NrIndeksuStrategy obsługuje walidację numeru indeksu. Jej zadaniem jest sprawdzenie, czy pole ma postać ośmiocyfrowego łańcucha znaków, zgodnie z wymaganiem podanym w instrukcji.

Ten fragment rozwiązania pokazuje, że w obrębie reguł mogą być obsługiwane nie tylko bardzo ogólne ograniczenia. Dzięki strategii walidacyjnej można łatwo dopisać kolejne zasady dotyczące specyficznych identyfikatorów, kodów, numerów lub innych danych o ustalonym formacie.

2.6.5. EmailStrategy

Klasa EmailStrategy odpowiada za walidację adresu e-mail. Jej zadaniem jest sprawdzenie, czy tekst w polu przypomina poprawny adres poczty elektronicznej i nie zawiera błędów składniowych.

Jest to przykład reguły, która w praktyce często pojawia się w różnych aplikacjach. Dzięki wydzieleniu jej do osobnej strategii można łatwo modyfikować sposób sprawdzania adresu e-mail bez wpływu na resztę mechanizmu walidacji.

2.7. Klasa ValidationStrategyFactory w zadaniu podstawowym

W projekcie podstawowym klasa ValidationStrategyFactory przechowuje mapę powiązań pomiędzy klasami adnotacji a strategiami. Kluczami mapy są klasy adnotacji, a wartościami klasy implementujące ValidationStrategy.

W bloku static znajdują się jawne wpisy, takie jak skojarzenie `NotNull.class` z obiektem `new NotNullStrategy()`. Analogicznie do mapy dopisywane są pozostałe strategie, co umożliwia późniejsze pobieranie właściwej implementacji na podstawie adnotacji znalezionej dla danego pola.

To rozwiązanie, chociaż proste i czytelne, ma pewne ograniczenie: każda nowa adnotacja i każda nowa strategia wymaga dopisania kolejnego powiązania w fabryce, co zwiększa liczbę miejsc, które muszą zostać zmodyfikowane.

Właśnie dlatego zadanie dodatkowe rozwiązuje ten problem. Celem tego rozszerzenia było pozostawienie fabryki, ale w taki sposób, żeby nie było konieczności ręcznego uzupełniania mapy.


```

1 package com.validation.strategy;
2
3 import java.lang.annotation.Annotation;
4 import java.util.HashMap;
5 import java.util.Map;
6 import com.validation.annotation.NotNull;
7 import com.validation.annotation.Size;
8 import com.validation.annotation.Email;
9 import com.validation.annotation.NotEmpty;
10 import com.validation.annotation.NrIndeksu;
11
12 public class ValidationStrategyFactory {
13     // fragment Class<? extends Annotation> określa, że kluczem mapy mogą być
14     // tylko klasy, które rozszerzają lub implementują interfejs Annotation
15     private static final Map<Class<? extends Annotation>, ValidationStrategy>
16         strategies = new HashMap<>();
17
18     static {
19         strategies.put(NotNull.class, new NotNullStrategy());
20         strategies.put(NotEmpty.class, new NotEmptyStrategy());
21         strategies.put(Size.class, new SizeStrategy());
22         strategies.put(NrIndeksu.class, new NrIndeksuStrategy());
23         strategies.put(Email.class, new EmailStrategy());
24     }
25
26     private ValidationStrategyFactory() {
27     }
28
29     public static ValidationStrategy getStrategy(Annotation annotation) {
30         return strategies.get(annotation.annotationType());
31     }
32 }

```

Rysunek 4 - kod klasy ValidationStrategyFactory w zadaniu podstawowym - czerwonym prostokątem zaznaczony blok statystyczny z ręcznymi wpisami do mapy

2.8. Klasa ValidationException i obsługa błędów walidacji

Klasa wyjątku rozszerza RuntimeException i może przechowywać tekst opisujący pojedynczy błąd albo kilka błędów zebranych podczas walidacji.

Zaletą takiego rozwiązania jest uproszczenie kodu wywołującego. Zamiast po każdej operacji walidacyjnej sprawdzać wyniki i reagować na nie przy pomocy dodatkowych instrukcji warunkowych, wystarczy przechwycić wyjątek w jednym miejscu programu.

To podejście realizuje założenia z instrukcji laboratoryjnej, ponieważ zamiast kodów błędów lub wartości specjalnych używany jest wyjątek ValidationException. Jeśli dane są błędne, dalsze przetwarzanie jest przerywane, a pełna informacja o wykrytych problemach jest przekazywana w postaci wyjątku.

2.9. Klasa Validator i wykorzystanie refleksji

Najważniejszą klasą całego rozwiązania jest Validator. To właśnie ona odpowiada za wykonanie walidacji niezależnie od konkretnej klasy modelowej. Dzięki refleksji validator nie musi znać na stałe pól klasy Student ani nazw zastosowanych adnotacji.

Metoda `validate(Object object)` pobiera klasę przekazanego obiektu i przechodzi po wszystkich polach tej klasy. Następnie dla każdego pola pobiera jego wartość oraz zestaw obecnych adnotacji. W dalszej kolejności walidator wykorzystuje fabrykę strategii do odnalezienia odpowiedniej logiki walidacyjnej dla każdej napotkanej adnotacji.

Takie rozwiązanie jest uniwersalne. Ta sama klasa `Validator` mogłaby zostać użyta także dla innych klas, o ile ich pola byłyby opisane tym samym zestawem adnotacji walidacyjnych. Dzięki refleksji i wzorcowi Strategia mechanizm nie jest związany tylko z jedną klasą `Student`.

2.9.1. Pobieranie pól klasy

Jednym z pierwszych kroków w metodzie `validate` jest pobranie pól klasy za pomocą `getDeclaredFields()`. Dzięki temu walidator uzyskuje dostęp do pełnej listy pól zadeklarowanych w klasie danego obiektu.

To wygodne rozwiązanie, ponieważ nie trzeba ręcznie wskazywać pól do walidacji. Wystarczy, że klasa obiektu posiada pola oznaczone odpowiednimi adnotacjami, a walidator sam je odnajdzie podczas wykonywania programu.

Zaprezentowana jest tu też jedna z podstawowych zalet refleksji, czyli możliwość dynamicznej analizy struktury klasy bez potrzeby wcześniejszego programowania osobnych procedur.

2.9.2. Udostępnianie pól prywatnych

Użycie `field.setAccessible(true)` umożliwia odczyt również pól prywatnych. Dzięki temu walidator może pracować nawet wtedy, gdy pola danej klasy nie są publiczne. Jest to przydatne, ponieważ pola danych zwykle pozostają prywatne, a dostęp do nich odbywa się przez metody dostępowe.

Refleksja pozwala w tym miejscu tymczasowo ominąć ograniczenie widoczności, co jest uzasadnione, ponieważ walidator pełni w aplikacji rolę mechanizmu infrastrukturalnego.

2.9.3. Iteracja po adnotacjach

Po pobraniu wartości pola walidator przechodzi przez wszystkie adnotacje przypisane do danego pola. Dla każdej znalezionej adnotacji pyta fabrykę o odpowiadającą jej strategię walidacyjną.

To rozwiązanie pozwala na jednym polu umieścić kilka adnotacji i wszystkie zostaną obsłużone po kolei. Przykładowo email może być jednocześnie sprawdzany pod kątem niepustej wartości oraz poprawnej składni adresu.

Dzięki temu reguły nie wykluczają się wzajemnie, ale mogą być łączone w sposób deklaratywny bez komplikowania kodu.

2.9.4. Gromadzenie komunikatów błędów

Każda strategia zwraca `Optional<String>`. Jeśli strategia wykryje błąd, walidator dopisuje komunikat do listy błędów. Jeśli walidacja danego warunku powiedzie się, wynik pozostaje pusty i nic nie jest dopisywane.

Umożliwia to zebranie więcej niż jednego błędu podczas pojedynczego wywołania walidatora. Program może przeanalizować cały obiekt i podać pełną listę błędów, zamiast przerywać działanie po pierwszym wystąpieniu.

Na końcu wszystkie komunikaty są łączone w jeden tekst i przekazywane do wyjątku `ValidationException`. Z punktu widzenia użytkownika lub testującego pozwala to od razu uzyskać komplet takich problemów w obiekcie.

```
1 package com.validation.validator;
2
3 import java.lang.annotation.Annotation;
4 import java.lang.reflect.Field;
5 import java.util.ArrayList;
6 import java.util.List;
7 import java.util.Optional;
8 import com.validation.exception.ValidationException;
9 import com.validation.strategy.ValidationStrategy;
10 import com.validation.strategy.ValidationStrategyFactory;
11
12 public class Validator {
13
14     private Validator() {
15     }
16
17     public static void validate(Object object) throws ValidationException {
18         final List<String> errors = new ArrayList<>();
19
20         final Class<?> clazz = object.getClass();
21         // iterowanie po wszystkich polach danej klasy
22         for (Field field : clazz.getDeclaredFields()) {
23             field.setAccessible(true); // umożliwia dostęp również do pól prywatnych danej klasy
24             try {
25                 Object value = field.get(object);
26                 // iterowanie po wszystkich adnotacjach danego pola
27                 for (Annotation annotation : field.getAnnotations()) {
28                     ValidationStrategy strategy = ValidationStrategyFactory.getStrategy(annotation);
29                     if (strategy != null) {
30                         Optional<String> validationError = strategy.validate(field, value);
31                         if (validationError.isPresent()) errors.add(validationError.get());
32                     }
33                 }
34             } catch (IllegalAccessException e) {
35                 errors.add("Błąd dostępu do pola: " + field.getName());
36             }
37         }
38
39         if (!errors.isEmpty()) throw new ValidationException(String.join("\n", errors));
40     }
41 }
42 }
```

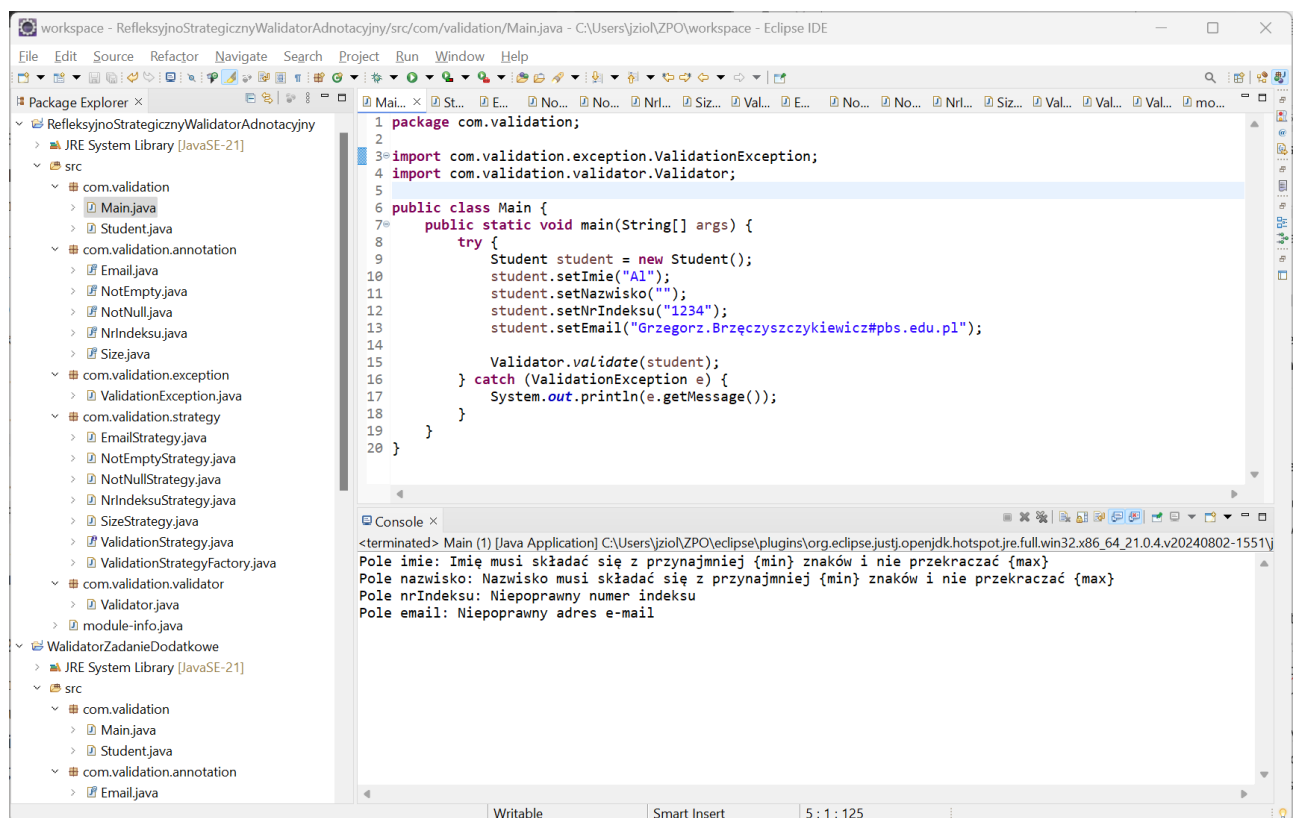
Rysunek 5 - kod klasy `Validator` - czerwonym prostokątem zaznaczone pętle iterująca po polach i adnotacjach

2.10. Testowanie rozwiązania podstawowego w klasie Main

Działanie walidatora jest testowane w klasie Main. Metoda main pełni funkcję prostego scenariusza testowego. Tworzony jest obiekt Student, po czym ustawiane są dla niego dane naruszające wybrane reguły walidacyjne, na przykład błędny adres e-mail, zbyt krótkie imię lub niepoprawny numer indeksu.

Następnie uruchamiana jest metoda Validator.validate(student).

Jeżeli dane są niepoprawne, wyjątek ValidationException jest przechwytywany w bloku catch, a jego treść zostaje wypisana w konsoli. Dzięki temu można łatwo zweryfikować, czy wszystkie strategie zostały uruchomione i czy komunikaty o błędach są zgodne z oczekiwaniami.



The screenshot shows the Eclipse IDE with the following components:

- Package Explorer:** Displays the project structure. The package `com.validation` contains `Main.java`, `Student.java`, and several strategy classes like `Email.java`, `NotEmpty.java`, `NotNull.java`, `NrIndeksu.java`, and `Size.java`. There are also exception and validator classes.
- Editor:** Shows the code of `Main.java`. The code defines a `Main` class with a `main` method. Inside `main`, a `Student` object is created and its fields are set: `imie` to "A1", `nazwisko` to "", `nrIndeksu` to "1234", and `email` to "Grzegorz.Bręczyszczykiewicz#pbs.edu.pl". Then, `Validator.validate(student)` is called. A `catch` block for `ValidationException` prints the error message to the console.
- Console:** Shows the output of the program. It indicates that the application terminated and then prints the error messages for each field: "Pole imię: Imię musi składać się z przynajmniej {min} znaków i nie przekraczać {max}", "Pole nazwisko: Nazwisko musi składać się z przynajmniej {min} znaków i nie przekraczać {max}", "Pole nrIndeksu: Niepoprawny numer indeksu", and "Pole email: Niepoprawny adres e-mail".

Rysunek 6 – widoczny kod klasy Main oraz wynik wykonania programu w konsoli (po wprowadzeniu błędnych wartości do walidacji)

2.11. Zadanie dodatkowe- założenie i cel

Zadanie dodatkowe miało na celu utworzenie sytuacji, w której nie byłoby potrzeby ręcznego deklarowania w fabryce powiązań pomiędzy adnotacjami walidacyjnymi a strategiami. W większych projektach takie rozwiązanie mogłoby stać się mniej bardziej podatne na pomyłki.

W projekcie dodatkowym celem było więc osiągnięcie większej elastyczności. Fabryka miała sama odnajdywać klasy strategii, odczytywać z nich metadane i automatycznie uzupełniać mapę. Dzięki temu system można łatwiej rozbudowywać.

2.11.1 Adnotacja @ValidationFor

Najważniejszym elementem zadania dodatkowego jest nowa adnotacja @ValidationFor. Służy ona do oznaczania klas strategii i wiązania ich z odpowiednią adnotacją walidacyjną.

W przeciwieństwie do wcześniejszych adnotacji nakładanych na pola klasy Student, @ValidationFor ma zastosowanie do klas strategii, dlatego używa @Target(ElementType.TYPE). Jej parametr value przechowuje klasę adnotacji, którą dana strategia ma obsługiwać.

To rozwiązanie przenosi informację o powiązaniu z fabryki do samej strategii. Program odczytuje relację „adnotacja - strategia” z metadanych przypisanych bezpośrednio do klasy strategii, zamiast zapisywać ją w osobnej mapie ręcznie.

```
1 package com.validation.annotation;
2
3 import java.lang.annotation.Annotation;
4 import java.lang.annotation.ElementType;
5 import java.lang.annotation.Retention;
6 import java.lang.annotation.RetentionPolicy;
7 import java.lang.annotation.Target;
8
9 @Retention(RetentionPolicy.RUNTIME)
10 @Target(ElementType.TYPE)
11 public @interface ValidationFor {
12     Class<? extends Annotation> value();
13 }
```

Rysunek 7 - definicja adnotacji @ValidationFor

2.11.2 Oznaczanie strategii adnotacją @ValidationFor

Po wprowadzeniu @ValidationFor każda klasa strategii w projekcie dodatkowym została odpowiednio oznaczona.

Przykładowo NotNullStrategy może być opisana jako @ValidationFor(NotNull.class), analogiczne oznaczenia otrzymują też pozostałe klasy strategii.

Nie ma już potrzeby ręcznego dopisywania klasy strategii do mapy w innej klasie, bo taka klasa strategii „wie”, jaką adnotację obsługuje. Informacja o powiązaniu jest przechowywana obok implementacji danej strategii.

Jest to również rozwiązanie bardziej odporne na błędy konfiguracyjne. Jeśli powstanie nowa strategia i zostanie poprawnie oznaczona, fabryka będzie mogła odnaleźć ją automatycznie bez dodatkowych zmian w kodzie.

2.11.3 Automatyczne wyszukiwanie strategii w ValidationStrategyFactory

W projekcie dodatkowym ValidationStrategyFactory nadal przechowuje mapę strategii, ale sposób jej wypełniania uległ zmianie. Zamiast wpisów dodawanych ręcznie w bloku statycznym, fabryka automatycznie wyszukuje klasy oznaczone @ValidationFor i na tej podstawie tworzy mapę powiązań.

Najpierw odnajdywane są klasy strategii z odpowiednią adnotacją. Następnie z każdej takiej klasy odczytywana jest wartość value z adnotacji ValidationFor, czyli klasa obsługiwanej adnotacji walidacyjnej. W kolejnym kroku tworzona jest instancja strategii i zapisywana w mapie pod odpowiednim kluczem.

Dzięki temu fabryka nie musi już znać z góry wszystkich konkretnych klas strategii. Jej rola ogranicza się do inicjalizacji mapy na podstawie odnalezionych metadanych, co jest rozwiązaniem łatwiejszym do ewentualnego rozszerzania.

```
1 package com.validation.strategy;
2
3 import java.lang.annotation.Annotation;
4 import java.util.HashMap;
5 import java.util.Map;
6 import java.util.Set;
7
8 import org.reflections.Reflections;
9
10 import com.validation.annotation.ValidationFor;
11
12 public class ValidationStrategyFactory {
13
14     private static final Map<Class<? extends Annotation>, ValidationStrategy>
15         strategies = new HashMap<>();
16
17     static {
18         registerStrategies();
19     }
20
21     private ValidationStrategyFactory() {
22     }
23
24     private static void registerStrategies() {
25         Reflections reflections = new Reflections("com.validation.strategy");
26         Set<Class<?>> classes = reflections.getTypesAnnotatedWith(ValidationFor.class);
27
28         for (Class<?> clazz : classes) {
29             if (ValidationStrategy.class.isAssignableFrom(clazz)) {
30                 ValidationFor annotation = clazz.getAnnotation(ValidationFor.class);
31
32                 try {
33                     ValidationStrategy strategy =
34                         (ValidationStrategy) clazz.getDeclaredConstructor().newInstance();
35                     strategies.put(annotation.value(), strategy);
36                 } catch (Exception e) {
37                     throw new RuntimeException("Błąd rejestracji strategii: " + clazz.getName(), e);
38                 }
39             }
40         }
41     }
42
43     public static ValidationStrategy getStrategy(Annotation annotation) {
44         return strategies.get(annotation.annotationType());
45     }
46 }
```

Rysunek 8 - kod zmodyfikowanej klasy ValidationStrategyFactory w zadaniu dodatkowym

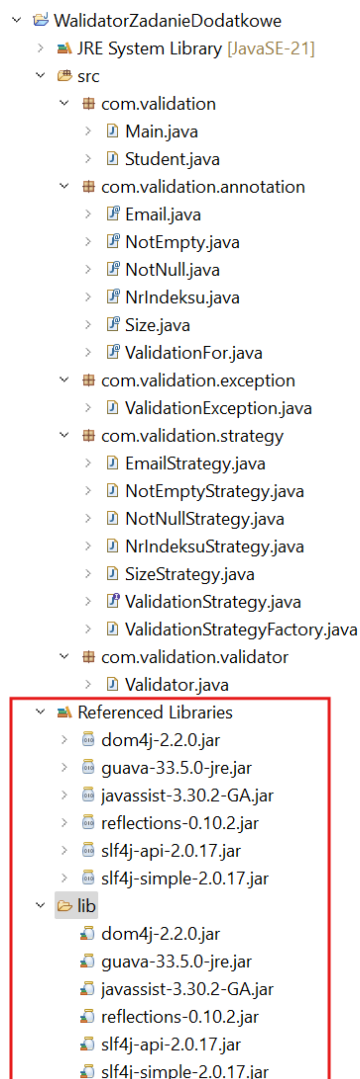
2.11.4 Biblioteka Reflections i konfiguracja projektu dodatkowego

W projekcie dodatkowym używana jest biblioteka Reflections, aby zrealizować automatyczne wyszukiwanie klas strategii. Umożliwia ona skanowanie wskazanego pakietu i odnajdywanie klas oznaczonych konkretną adnotacją.

Ze względu na to, że projekt nie korzystał z Maven-a, pobrałem ręcznie biblioteki jako pliki JAR i dodałem do projektu. W tym celu utworzyłem katalog lib, do którego zostały skopiowane wymagane biblioteki, a następnie dodane do Classpath projektu.

Oprócz samej biblioteki reflections potrzebne były również zależności związane z jej działaniem. W praktyce oznaczało to dołączenie kilku plików JAR, a następnie sprawdzenie, czy projekt poprawnie rozpoznaje import `org.reflections.Reflections`.

To była różnica względem projektu podstawowego, który nie wymagał żadnych bibliotek zewnętrznych. Dzięki temu projekt dodatkowy stał się bardziej zaawansowany, ale jednocześnie wymagał większej uwagi na etapie konfiguracji środowiska.



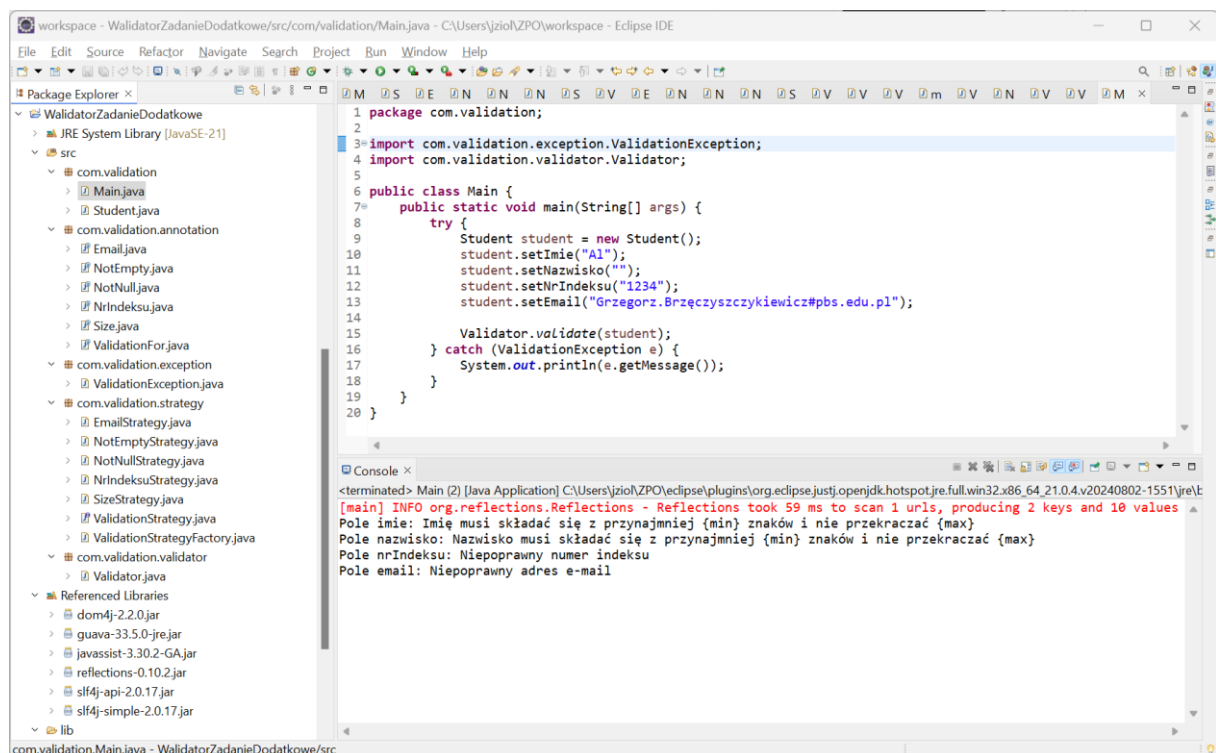
Rysunek 9 - struktura projektu zadania dodatkowego w Package Explorer z zaznaczonymi: folderem lib i Referenced Libraries

2.11.5. Problemy konfiguracyjne w Eclipse i ich rozwiązanie

Podczas konfiguracji projektu dodatkowego pojawiły się typowe związane z bibliotekami zewnętrznymi i systemem modułów Javy. Jednym z nich był plik module-info.java, który w zwykłym projekcie Eclipse utrudniał poprawne rozpoznawanie bibliotek dołączonych ręcznie jako JAR-y.

Ostatecznie plik module-info.java został usunięty z projektu dodatkowego. Po tej zmianie import org.reflections.Reflections przestał być oznaczany jako błędny, a projekt zaczął się poprawnie kompilować i uruchamiać. Było to rozwiązanie praktyczne i bezpieczne, ponieważ instrukcja wymagała działania mechanizmu walidacji i automatycznego wyszukiwania strategii, a nie konieczności użycia modularnego projektu Java.

Kolejny problem dotyczył komunikatów SLF4J o braku providera logowania. Został on rozwiązany przez dołączenie odpowiedniego JAR-a, dzięki czemu zniknęły ostrzeżenia przy uruchamianiu programu. Po tej konfiguracji biblioteka Reflections działała prawidłowo i wypisywała w konsoli jedynie informacyjny komunikat o czasie skanowania pakietu, co było poprawnym zachowaniem.



```
1 package com.validation;
2
3 import com.validation.exception.ValidationException;
4 import com.validation.validator.Validator;
5
6 public class Main {
7     public static void main(String[] args) {
8         try {
9             Student student = new Student();
10            student.setImie("A1");
11            student.setNazwisko("");
12            student.setNrIndeksu("1234");
13            student.setEmail("Grzegorz.Brzeczyszczykiewicz@pbs.edu.pl");
14
15            Validator.validate(student);
16        } catch (ValidationException e) {
17            System.out.println(e.getMessage());
18        }
19    }
20 }
```

<terminated> Main [2] [Java Application] C:\Users\jzio\ZPO\workspace\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64_21.0.4.v20240802-1551\jre\l
[main] INFO org.reflections.Reflections - Reflections took 59 ms to scan 1 urls, producing 2 keys and 10 values
Pole imie: Imię musi składać się z przynajmniej {min} znaków i nie przekraczać {max}
Pole nazwisko: Nazwisko musi składać się z przynajmniej {min} znaków i nie przekraczać {max}
Pole nrIndeksu: Niepoprawny numer indeksu
Pole email: Niepoprawny adres e-mail

3. Wnioski końcowe/Podsumowanie

W trakcie realizacji laboratorium przećwiczyłem istotne elementy zaawansowanego programowania obiektowego w Javie. Najważniejsze z nich to tworzenie własnych adnotacji, korzystanie z refleksji do analizy struktury obiektów, obsługa wyjątków oraz praktyczne zastosowanie wzorca Strategia.

Zadanie podstawowe pozwoliło zrozumieć, jak oddzielić deklarację reguł walidacyjnych od kodu pozostałych klas. Dzięki temu klasa Student nie zawiera logiki sprawdzania własnych pól, ale tylko deklaruje ograniczenia przy pomocy adnotacji. Rzeczywiste wykonanie walidacji odbywa się dopiero w osobnych klasach strategii oraz w walidatorze.

Zadanie dodatkowe była bardziej wymagające, ponieważ wymagało nie tylko modyfikacji architektury, ale też poprawnej konfiguracji bibliotek zewnętrznych w Eclipse. Jednocześnie to właśnie pokazało najciekawszy aspekt całego laboratorium, czyli możliwość dalszej automatyzacji rozwiązania i budowania systemu, który łatwiej rozszerzać.

Realizacja obu projektów pokazała również różnicę między rozwiązaniem prostym a bardziej zaawansowanym i elastycznym. Projekt podstawowy jest łatwiejszy do zrozumienia i uruchomienia, natomiast projekt dodatkowy lepiej pokazuje kierunek, w którym można rozwijać dalej tego typu mechanizmy.

Podsumowując, cel laboratorium został osiągnięty. Został przygotowany działający mechanizm walidacji danych wejściowych z użyciem adnotacji, refleksji i wzorca Strategia, a następnie rozszerzony o automatyczne wykrywanie strategii walidacyjnych przy użyciu adnotacji @ValidationFor i skanowania klas.